# p.637 해설 — 초록과 서론

> 이 PDF는 1974년 인쇄본을 스캔한 것이라 글자가 흐릿할 수 있습니다. 저자 이름 "ICAHN"은 스캔 오류로, 실제 이름은 **Robert E. Kahn**입니다.

① **초록** — **서로 다른 패킷 교환 네트워크** 사이에서 자원을 공유하는 프로토콜을 제안한다. 다루는 문제: 네트워크마다 다른 **패킷 크기**, **전송 실패**, **순서 맞추기**, **흐름 제어**, **종단 간 오류 검사**, 프로세스 간 **논리적 연결의 생성과 해제**. 라우팅·과금·타임아웃 같은 남은 문제도 드러낸다. → 🔎 초록의 문제 목록에 번호를 붙여 두세요. 논문이 이 순서대로 하나씩 해결합니다.

② **서론 첫 문단** — 패킷 교환망의 주목적은 **컴퓨터 자원 공유**. 기존 프로토콜은 **같은 네트워크 안의** 통신만 다뤘고, 이 논문은 **다른 네트워크 사이**를 다룬다.

③ **논문 구성 예고** — 네트워크 사이의 인터페이스인 **GATEWAY** → 주소·형식·버퍼·순서·흐름 제어·오류 제어 → 프로세스 간 통신 메커니즘.

④ **오른쪽 단: 패킷 교환망의 구성** — **HOST**(컴퓨터 자원), **패킷 스위치**, 그 사이의 통신 매체. 각 HOST 안의 **프로세스**가 데이터의 최종 출발지와 목적지이다. → 오늘날 용어로: HOST = 컴퓨터, 패킷 스위치 = 라우터, 프로세스 = 실행 중인 프로그램.

| 1974년 용어 | 오늘날 |
|---|---|
| HOST | 호스트(컴퓨터) |
| packet switch | 라우터/스위치 |
| GATEWAY | 라우터(망 사이) |
| process | 프로세스(앱) |

# p.638 해설 — 네트워크들은 어떻게 다른가, 그리고 GATEWAY

① **네트워크 간 차이 (번호 목록)** — 네트워크마다 다른 것들:

1. 주소 체계
2. 최대 패킷 크기
3. 전송 지연(성공·실패 판정 시간)
4. 손실·손상 패킷 처리 방식
5. 상태 정보·라우팅·고장 감지 방식

② **설계 철학** ⭐ — 모든 차이를 경계(인터페이스)에서 해결할 수 있다면 편하겠지만, 경제적·기술적 이유로 인터페이스는 **가능한 한 단순하고 신뢰성 있게**, 주로 **데이터 전달**만 하도록 한다. → 🔎 *as simple and reliable as possible* 에 밑줄 — 훗날 **종단 간 원칙(end-to-end principle)** 의 씨앗.

③ **공통 프로토콜의 필요성** — HOST·프로세스 수준 프로토콜을 경계에서 변환하는 대신, **모든 HOST가 하나의 공통 프로토콜**을 쓴다. 그렇지 않으면 모든 HOST가 (잠재적으로 무한히 많은) 모든 프로토콜을 구현해야 한다. → 수학적 관점: $n$개 네트워크를 쌍마다 변환하면 $O(n^2)$, 공통 표준 하나면 $O(n)$.

④ **과금(accounting)** — 소유자가 다른 네트워크를 연결하면 통과 트래픽에 대한 과금이 필요.

⑤ **GATEWAY 개념 & Fig. 1** — 네트워크 간 인터페이스에 **GATEWAY**라는 이름을 붙인다. Fig. 1은 패킷 스위치(PS)들로 이루어진 전형적 패킷 교환망.

# p.639 해설 — 패킷 형식과 분할(fragmentation)

① **Fig. 2** — 세 네트워크 A, B, C를 GATEWAY M, N이 연결. GATEWAY는 **양쪽 네트워크 모두의 HOST처럼** 행동한다.

② **Fig. 3: 인터넷 패킷 형식** ⭐ `[지역 헤더] [출발지] [목적지] [순서 번호] [바이트 수] [플래그] [본문] [체크섬]`

- 출발지/목적지: 전체 네트워크에서 **유일한 주소**
- 순서 번호 + 바이트 수: 도착 후 **순서 복원**, 고장 감지
- 체크섬: **종단 간** 검증. GATEWAY는 체크섬을 **다시 계산하지 않고** 그대로 전달! → 🔎 *The GATEWAY does not modify the text* 에 밑줄.

③ **캡슐화** — 각 네트워크는 자기 형식의 **지역 헤더**를 앞에 붙여 인터넷 패킷을 감싼다. → 오늘날의 **계층화와 캡슐화** 개념의 원형.

④ **분할(fragmentation)** — 모든 패킷을 아주 작게 제한하지 않는 한, GATEWAY가 큰 패킷을 **여러 조각으로 나눠야** 하고, 목적지는 조각을 **다시 조립**할 수 있어야 한다.

⑤ **최대 패킷 크기를 정하지 않는 이유 (번호 목록)**

1. 각 네트워크의 내부 매개변수를 서로 **격리**할 수 없게 됨
2. 새 기술(큰 메모리, 빠른 회선)이 나와도 **모두가 합의**해야 크기를 늘릴 수 있음
3. 암호화 등으로 전송 중 패킷 크기가 **커질 수** 있음 → "미래의 기술 변화를 막지 않는 설계" — 50년 뒤에도 인터넷이 작동하는 이유.

# p.640 해설 — 프로세스 수준 통신과 TCP의 탄생

① **앞 쪽 마무리** — GATEWAY에서 비용 차이를 각 네트워크 성능 최적화의 경제적 지렛대로 쓸 수 있다.

② **프로세스 수준 통신** — 프로세스들은 **전이중(full duplex)** 으로, 길이 제한 없는(그러나 유한한) **메시지**를 주고받는다. 한 글자도, 한 페이지도, 연속 데이터 스트림도 메시지로 표현 가능.

③ **TCP 등장** ⭐ — 각 HOST에는 프로세스를 대신해 메시지를 보내고 받는 **전송 제어 프로그램(Transmission Control Program, TCP)** 이 있다. → 🔎 *transmission control program (TCP)* — **TCP라는 이름이 처음 등장하는 곳!** 크게 표시하세요.

④ **세그먼트** — TCP는 메시지를 **세그먼트**로 쪼갤 수 있다. 이유: 수신자의 수용량 제한, 지역 네트워크의 최대 크기, 여러 프로세스 간 자원 공유. 세그먼트 길이는 **8비트 바이트의 정수배** — 워드 길이가 다른 기계들 사이를 단순화.

⑤ **다중화/역다중화** — TCP의 핵심 업무: 송신 시 여러 프로세스의 세그먼트를 모아 패킷으로, 수신 시 섞여 도착한 패킷을 올바른 프로세스로 분배.

⑥ **Case 1 vs Case 2** — 같은 목적지로 가는 세그먼트들을 **하나의 바이트 스트림**으로 이어 붙여 효율을 높일 것인가(Case 1), 아니면 각 패킷에 **세그먼트 하나씩** 담을 것인가(Case 2)? 저자들은 다음 쪽에서 단순성을 위해 Case 2를 택한다.

# p.641 해설 — 주소 체계: 네트워크 + TCP + 포트

① **왼쪽 단 위 (Case 2 선택)** — 재조립의 복잡성 때문에 **패킷 하나에 세그먼트 하나**를 담는 방식을 택함.

② **주소 형식** — 네트워크마다 TCP의 지역 주소 형식이 다르므로, 모든 GATEWAY와 TCP가 이해하는 **통일된 인터넷 주소 공간**이 필수.

③ **포트(port) 개념** ⭐ — 한 프로세스가 여러 메시지 흐름을 구분할 수 있도록 **포트**를 도입. 포트 = 프로세스에 연결된 하나의 전이중 메시지 흐름. → 오늘날 웹 서버의 80/443번 포트가 바로 이 개념.

④ **TCP 주소 (Fig. 4)**

$$\underbrace{\text{네트워크 ID}}_{8\text{비트}} \,\Big|\, \underbrace{\text{TCP 식별자}}_{16\text{비트}}$$

- 8비트 → 최대 $2^8 = 256$개 네트워크 ("가까운 미래엔 충분해 보인다")
- 16비트 → 네트워크당 $2^{16} = 65{,}536$개 TCP → 🔎 *This size seems sufficient for the forseeable future* 에 밑줄 — 역사의 아이러니! 이후 IPv4(32비트), IPv6(128비트)로 확장.

⑤ **라우팅** — GATEWAY는 목적지 **네트워크 ID만** 보고 경로를 결정. 목적지 네트워크가 직접 연결되어 있으면 아래 16비트로 지역 주소를 만들고, 아니면 다음 GATEWAY로 전달. → **계층적 주소** 덕분에 GATEWAY는 모든 호스트를 알 필요가 없다.

⑥ **포트 주소** — 16비트 포트 번호. 보내는 쪽은 받는 쪽이 포트 번호를 내부적으로 어떻게 쓰는지 알 필요 없음.

# p.642 해설 — 세그먼트, 순서 번호, 플래그

① **Fig. 5** — 메시지 → 세그먼트 → 패킷 생성 과정. 지역 헤더(LH), 인터넷 헤더(IH), 프로세스 헤더(PH), 체크섬(CK).

② **Fig. 6: 세그먼트 형식** — 출발지 포트(32) · 목적지 포트(32) · **윈도우**(16) · **ACK**(16) · 본문. → 오늘날 TCP 헤더의 원형. 윈도우와 ACK가 이미 들어 있음!

③ **순서 번호 규칙** ⭐ — 세그먼트 본문 **첫 바이트의 위치**를 패킷의 순서 번호로 쓴다(바이트 단위 번호). → 🔎 이렇게 하면 **중간 조각이 빠져 있어도** 도착한 데이터의 정확한 위치를 알 수 있다.

④ **Fig. 7** — 순서 번호는 **통신하는 포트 쌍마다** 독립적으로 매겨진다.

⑤ **Fig. 8: 플래그 필드**

| 플래그 | 의미 |
|---|---|
| ES | 세그먼트의 끝 |
| EM | 메시지의 끝 |
| REL | 포트 사용 해제 |
| SYN | 순서 번호 동기화 |
| → SYN은 오늘날 TCP 연결 시작(**3-way handshake**)의 그 SYN. | |

⑥ **체크섬** — 세그먼트마다 본문과 프로세스 헤더에 대해 계산. GATEWAY가 세그먼트를 더 쪼갤 수 있으므로 ES/EM 플래그로 끝을 알린다.

# p.643 해설 — 재전송, 중복 감지, 슬라이딩 윈도우

① **재전송** — "어떤 전송도 100% 신뢰할 수 없다." **타임아웃 + 긍정 확인 응답(ACK)**: 보낸 뒤 ACK가 안 오면 다시 보낸다. → 🔎 *No transmission can be 100 percent reliable* 에 밑줄.

② **중복 감지의 어려움** — 재전송하면 수신자는 같은 패킷을 두 번 받을 수 있다. 순서 번호가 **유한**하고 재사용되므로, 새 전송과 재전송을 구분해야 한다.

③ **윈도우 전략** ⭐ **(프랑스 CYCLADES, ARPANET에서 영감)**

- 순서 번호 공간: $0 \sim n-1$ (순환)
- 송신자는 수신자의 **왼쪽 윈도우 끝** $L$ 로부터 $w$ 바이트 이내만 전송: $[L,\ L+w-1]$
- 확인 응답을 받으면 윈도우가 오른쪽으로 **미끄러짐** → 슬라이딩 윈도우

④ **수신 규칙**

1. 순서 번호 = $L$ 인 패킷 → 받아들이고 $L$ 전진, ACK 전송
2. $L$ 보다 왼쪽(이미 받은 것) → **중복**, 버리고 현재 $L$을 다시 ACK
3. 윈도우 안이지만 $L$ 이 아닌 것 → 보관하거나 버림(순서 어긋남)

⑤ **핵심 조건** — 윈도우 크기는 **순서 번호 공간의 절반보다 작아야** 한다: $w < n/2$. → ✍️ 왜일까? 직접 증명해 보세요. (힌트: 순서 번호가 한 바퀴 돌았을 때 새 패킷과 옛 재전송 패킷을 구분할 수 있어야 함)

⑥ **Fig. 9** — 순서 번호 공간과 윈도우의 그림.

# p.644 해설 — TCP 입출력과 흐름 제어

① **TCP 입출력 처리** — 패킷이 도착하면 주소를 검증하고 큐에 넣는다. 버퍼가 가득 차면 새 도착 패킷은 **버려도 된다** — 확인 응답이 안 된 패킷은 어차피 **재전송**되므로. → 신뢰성을 네트워크가 아니라 **양 끝의 재전송**으로 보장하는 설계.

② **송신 버퍼링** — 송신 쪽은 프로세스 버퍼에 데이터가 있으므로 버퍼가 적게 필요. 구체적 버퍼 방식은 규정하지 않음.

③ **흐름 제어** ⭐ — 수신자가 ACK와 함께 **윈도우 크기**를 알려 송신자의 전송량을 조절한다. 수신자는 남은 버퍼 공간에 따라 윈도우를 **자유롭게 늘이거나 줄일** 수 있다. → 🔎 오늘날 TCP의 **수신 윈도우(rwnd)** 와 같은 개념. 처리량 ≈ 윈도우 / 왕복시간(RTT).

④ **장단점** — 이 방식은 단순하고 유연하며 동기화 문제가 없다. 하지만 수신자가 윈도우를 줄였는데 이미 보낸 패킷이 도착하면 **버려지고 재전송**이 필요할 수 있다.

⑤ **도착 패킷 처리 절차** — 올바른 프로세스로 넘기려 할 때 실패할 수 있는 경우:

1. 목적지 포트가 그 출발지로부터 받을 준비가 안 됨
2. 버퍼 공간 부족
3. 순서 번호가 다음에 전달할 번호와 다름 → 각 경우 패킷을 버리거나 보관하는 규칙을 정한다.

# p.645 해설 — 전송 제어 블록(TCB)

① **Fig. 10/11: TCB 형식** — TCP가 연결마다 관리하는 **상태 정보 묶음**.

| # | 필드 | 의미 |
|---|---|---|
| 1 | 출발지 주소 | 네트워크/HOST/TCP/포트 전체 |
| 2 | 목적지 주소 | 상대방 전체 주소 |
| 3 | 다음 패킷 순서 번호 | 다음에 보낼 번호 |
| 4 | 현재 버퍼 크기 | 남은 공간 |
| 5 | 다음 쓰기 위치 | 버퍼에 쓸 곳 |
| 6 | 다음 읽기 위치 | 버퍼에서 읽을 곳 |
| 7 | 끝 읽기 위치 | 전송을 멈출 위치 |
| 8 | 재전송 횟수 / 최대 재전송 횟수 | 몇 번 다시 보냈는지 (포기 기준) |
| 9 | 타임아웃/플래그 | 재전송까지 기다릴 시간, 동기화·상태 |
| 10 | 현재 ACK / 윈도우 | 아직 확인 안 된 첫 바이트, 윈도우 크기 |

다음 순서 번호 ≤ 현재 ACK + 윈도우, 그리고 **현재 ACK + 순서 번호 공간의 절반**을 넘지 않아야 한다 (중복 감지가 헷갈리지 않도록). 수신 쪽 **RCB**도 거의 같고, 여러 패킷으로 나뉜 세그먼트의 체크섬을 부분적으로 계산해 둔다.

→ 🔎 컴퓨터 과학 관점: **연결 = 상태(state)를 가진 객체**. 운영체제의 프로세스 제어 블록(PCB)과 같은 발상.

② **Fig. 12: 송신 버퍼 배치** — 순환(ring) 버퍼 안에서 "보냈지만 ACK 안 됨", "보낼 차례", "빈 공간" 구간을 포인터로 관리.

③ **연결과 연관(CONNECTIONS AND ASSOCIATIONS)** — 이 절부터 두 프로세스가 통신을 **시작하고 끝내는** 방법을 다룬다.

# p.646 해설 — 연결 없는 프로토콜과 연관 설정

**① 연관(association) 설정** — 미리 "연결"을 만들지 않고, 첫 패킷의 **SYN 플래그**로 순서 번호를 맞추며 통신을 시작한다.

**② 첫 패킷 규칙 (번호 목록)**

1. 첫 패킷은 **완전한 세그먼트**여야 한다
2. 첫 패킷에 대한 ACK를 보내 상대가 동기화되었음을 알린다 → 🔎 이 "SYN을 보내고 ACK를 받는" 절차가 이후 **3-way handshake (SYN → SYN-ACK → ACK)** 로 정교해진다.

**③ 초기 순서 번호** — 이전 연결의 지연된 패킷과 헷갈리지 않도록 순서 번호를 신중하게 정해야 한다. → 확률적 관점: 무작위 초기 순서 번호는 이후 **보안**(세션 가로채기 방지)에도 중요해진다.

**④ 연결 없는(connection-free) 방식의 장점** — 상태를 미리 만들지 않아 단순. 단, 중복 패킷 문제는 순서 번호와 윈도우로 해결해야 한다.

→ 읽기 팁: 이 쪽은 스캔 품질 때문에 읽기 어려울 수 있습니다. **SYN, ACK, 첫 패킷**이라는 단어가 나오는 문장만 찾아 밑줄 치세요.

# p.647 해설 — 연관 종료와 결론

① **연관 종료** — 메시지가 여러 패킷으로 쪼개질 수 있으므로 REL 플래그는 **EM과 함께 설정된 패킷에서만** 유효. 종료 시 TCP는 **ES + EM + REL** 플래그를 단 패킷을 보낸다.

② **조기 종료 방지** — 순서 번호 덕분에 아직 도착하지 않은 패킷이 있으면 수신 TCP가 알아챈다. 수신 TCP는 REL에 대해 **REL 확인 응답**을 보내 양쪽이 모두 종료를 인지하게 한다. → 오늘날 TCP의 **FIN / FIN-ACK** 종료 절차의 원형.

③ **한 번에 한 메시지 모드** — 첫 패킷에 SYN, 마지막 패킷에 ES·EM·REL을 달면 "메시지 하나씩" 주고받는 방식이 된다.

④ **남은 문제** — 송신자가 ACK를 못 받으면 수신자가 **중복을 잘못 받아들일** 수 있다는 지적이 있다. 완벽한 신뢰성은 여전히 어려운 문제. → ✍️ 이것은 분산 시스템의 유명한 **"두 장군 문제"** 와 연결됩니다. 찾아보세요!

⑤ **결론** — 서로 다른 네트워크 사이의 통신을 위한 프로토콜을 제시했다. 남은 과제: 라우팅, 과금, 타임아웃 설정 등.

# p.648 해설 — 참고문헌과 논문 정리

이 쪽은 참고문헌입니다. 당시 네트워크 연구의 지형을 보여줍니다.

| 참고문헌 | 의미 |
|---|---|
| ARPANET 관련 (Carr, Crocker, Cerf) | 미국 최초의 패킷 교환망, HOST-HOST 프로토콜 |
| CYCLADES (프랑스) | 윈도우 전략의 출처, 데이터그램 개념 |
| NPL (영국, Scantlebury 등) | 패킷 교환 초기 연구 |
| Kahn & Crowther (1972) | 흐름 제어 — 저자 Kahn의 이전 연구 |

### ✍️ 논문 전체 정리 질문

1. 이 논문에서 오늘날의 **IP**에 해당하는 부분과 **TCP**에 해당하는 부분을 나눠 보자. (힌트: GATEWAY가 하는 일 vs 양 끝 HOST가 하는 일)
2. "네트워크는 단순하게, 신뢰성은 양 끝에서" — 이 원칙이 왜 인터넷을 50년 동안 확장 가능하게 했을까?
3. 슬라이딩 윈도우에서 $w < n/2$ 조건을 직접 증명해 보자.
4. AI 연결: 거대 모델의 **분산 학습**(여러 GPU가 기울기를 주고받음)에서 이 논문의 어떤 개념(흐름 제어, 재전송, 순서 번호)이 필요할까?